OS-Orange Problem-U4

SRN:PES2UG24CS088

Section:4B

Github link: https://github.com/Aryan-20-2006/OS-U4-Orange

1A

```
aryan@Aryan:/mnt/c/Users/Aryan/OneDrive/Desktop/OS-U4-Orange$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

1B

```
aryan@Aryan:/mnt/c/Users/Aryan/OneDrive/Desktop/OS-U4-Orange$ find .pes/objects -type f
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
```

2A

```
aryan@Aryan:/mnt/c/Users/Aryan/OneDrive/Desktop/OS-U4-Orange$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

2B

```
aryan@Aryan:/mnt/c/Users/Aryan/OneDrive/Desktop/OS-U4-Orange$ rm -rf .pes
./pes init
echo hello > t1.txt
./pes add t1.txt
./pes commit -m "tree sample"

COMMIT=$(cat .pes/refs/heads/main)
COMMIT_OBJ=.pes/objects/${COMMIT:0:2}/${COMMIT:2}
TREE=$(grep -ao 'tree [0-9a-f]\{64\}' "$COMMIT_OBJ" | head -1 | cut -d' ' -f2)

xxd .pes/objects/${TREE:0:2}/${TREE:2} | head -20
Initialized empty PES repository in .pes/
Committed: 53b3e63e6594... tree sample
00000000: 7472 6565 2034 3600 3130 3037 3535 2074  tree 46.100755 t
00000010: 312e 7478 7400 2cf8 d83d 9ee2 9543 b34a  1.txt.,..=...C.J
00000020: 8772 7421 fdec b7e3 f3a1 83d3 3763 9025  .rt!........7c.%
00000030: de57 6db9 ebb4                           .Wm...
```

3A

```
aryan@Aryan:/mnt/c/Users/Aryan/OneDrive/Desktop/OS-U4-Orange$ mkdir -p /tmp/pes-phase3
cp ./pes /tmp/pes-phase3/
cd /tmp/pes-phase3

./pes init
echo "hello" > file1.txt
echo "world" > file2.txt
./pes add file1.txt file2.txt
./pes status
Initialized empty PES repository in .pes/
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)
```

3B

```
aryan@Aryan:/tmp/pes-phase3$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776442794 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776442794 6 file2.txt
```

4A

```
aryan@Aryan:/tmp/pes-phase3$ rm -rf .pes
./pes init

echo "Hello" > hello.txt
./pes add hello.txt
./pes commit -m "Initial commit"

echo "World" >> hello.txt
./pes add hello.txt
./pes commit -m "Add world"

echo "Goodbye" > bye.txt
./pes add bye.txt
./pes commit -m "Add farewell"

./pes log
Initialized empty PES repository in .pes/
Committed: 54368a3ab433... Initial commit
Committed: 2318fb42b0c3... Add world
Committed: 19d19e5b780c... Add farewell
commit 19d19e5b780c9cbd3bc4a52553df79ddc8ac1aee8838540d9735883766ecfd85
Author: PES User <pes@localhost>
Date:   1776442886

    Add farewell

commit 2318fb42b0c30e339782c4a969460b6d122b01eed88e55f34ee89d6dc819b6d2
Author: PES User <pes@localhost>
Date:   1776442886

    Add worldell

commit 54368a3ab43360d90aa151f20675c766dde51f977461c76641ef9a128f54b16c
Author: PES User <pes@localhost>
Date:   1776442886

    Initial committial commiQ
```

4B

```
aryan@Aryan:/tmp/pes-phase3$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/19/d19e5b780c9cbd3bc4a52553df79ddc8ac1aee8838540d9735883766ecfd85
.pes/objects/23/18fb42b0c30e339782c4a969460b6d122b01eed88e55f34ee89d6dc819b6d2
.pes/objects/54/368a3ab43360d90aa151f20675c766dde51f977461c76641ef9a128f54b16c
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/a6/882ee4afdc855ea4e39f4e0fc77d90e73a6f72deb383e11934310a10a030f6
.pes/objects/bd/9b769bb4d9910c31b4f0e99e4375fffea35c2055130c248ff131f78c626442
.pes/objects/d8/f28203071e10a3bde605928368bc5b51871287a867d8d10382967099dde814
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/refs/heads/main
```

4C

```
aryan@Aryan:/tmp/pes-phase3$ cat .pes/refs/heads/main
cat .pes/HEAD
19d19e5b780c9cbd3bc4a52553df79ddc8ac1aee8838540d9735883766ecfd85
ref: refs/heads/main
```

Final integration screenshot

```
aryan@Aryan:/mnt/c/Users/Aryan/OneDrive/Desktop/OS-U4-Orange$ sed -i 's/\r$//' test_sequence.sh
make test-integration
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)
```

```
--- First Commit ---
Committed: 450c201df8a1... Initial commit

Log after first commit:
commit 450c201df8a1ba23e4916e649be943348b603991d3368ca758f74116873514bc
Author: Aryan Burman <PES2UG24CS088>
Date:   1776443147

    Initial commit


--- Second Commit ---
Committed: b2001e46bd91... Update file.txt

--- Third Commit ---
Committed: 3ef8427a4511... Add farewell

--- Full History ---
commit 3ef8427a4511f98127736fc8fab7a891759c161abc0836193e73b695fdd0fbc7
Author: Aryan Burman <PES2UG24CS088>
Date:   1776443147

    Add farewell

commit b2001e46bd91180c8724f71bb24588a43737f0ae00dce9c57b51d132b282fbc1
Author: Aryan Burman <PES2UG24CS088>
Date:   1776443147

    Update file.txt

commit 450c201df8a1ba23e4916e649be943348b603991d3368ca758f74116873514bc
Author: Aryan Burman <PES2UG24CS088>
Date:   1776443147
```

```
    Update file.txt

commit 450c201df8a1ba23e4916e649be943348b603991d3368ca758f74116873514bc
Author: Aryan Burman <PES2UG24CS088>
Date:   1776443147

    Initial commit


--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
3ef8427a4511f98127736fc8fab7a891759c161abc0836193e73b695fdd0fbc7

--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/3e/f8427a4511f98127736fc8fab7a891759c161abc0836193e73b695fdd0fbc7
.pes/objects/45/0c201df8a1ba23e4916e649be943348b603991d3368ca758f74116873514bc
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/b2/001e46bd91180c8724f71bb24588a43737f0ae00dce9c57b51d132b282fbc1
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafeaa3829f61f75b8d10d5350f9
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff

=== All integration tests completed ===
```

5.1 How to Implement pes checkout <branch>

What changes in .pes:

1. Read current HEAD.
2. Update HEAD to point to target branch (ref: refs/heads/<branch>).
3. Read target branch file to get commit hash.
4. Rebuild index to match target commit tree.

What changes in working directory:

1. Read target commit and root tree.

2. Recursively materialize files/directories from tree.
3. Update files that changed.
4. Delete tracked files not present in target tree.
5. Restore proper file modes (100644 vs 100755).

Why checkout is complex:

1. Recursive tree traversal.
2. File-vs-directory path conflicts.
3. Protecting uncommitted local changes.
4. Avoiding partial/half-applied checkouts if failure happens mid-way.

5.2 Detect Dirty Working Directory Conflicts

Conflict detection method (using index + object store):

1. For each tracked path in index, compare current file metadata with indexed metadata.
2. If metadata differs, recompute blob hash of working file.
3. Compare working blob hash with index blob hash.
4. If different, mark file as dirty.
5. Compute which paths would change when switching from current tree to target tree.

6. If any dirty path intersects with would-change paths, refuse checkout.

Why this works:

1. It blocks destructive checkouts only where local edits would be overwritten.
2. It allows safe checkout when dirty files are unrelated to branch differences.

5.3 Detached HEAD: Behavior and Recovery

What detached HEAD means:

1. HEAD points directly to a commit hash, not a branch ref.

If you commit in detached HEAD:

1. Commits are created normally.
2. HEAD moves forward to new commits.
3. No branch pointer is updated.

Risk:

1. These commits become “orphaned” if you switch away.
2. They may later be garbage-collected if unreachable.

Recovery:

1. Create a branch at current detached commit before switching.
2. If already switched, recover commit hash from reflog/history and create a branch pointing to it.

6.1 Algorithm for Garbage Collection (Unreachable Objects)

Mark-and-sweep approach:

1. Collect all refs as roots (all branch heads, tags if present).
2. Traverse from each root commit:
3. Mark commit reachable.
4. Mark referenced tree reachable.
5. Recursively mark subtrees.
6. Mark all referenced blobs.
7. Follow parent commits and continue.
8. After marking, scan object store.
9. Delete objects not in reachable set.

Data structure:

1. Hash set for reachable object IDs (fast membership).
2. Stack/queue for traversal worklist.

Scale estimate (100k commits, 50 branches):

1. Unique commits visited is usually near 100k (not 5 million), because branches share history.
2. Plus all reachable unique trees and blobs (depends on repo churn).

6.2 Why Concurrent GC with Commit is Dangerous

Race condition example:

1. A commit operation writes new blobs/trees first.
2. Before ref update, these objects are not yet reachable from branch refs.
3. GC runs concurrently and sees them as unreachable.
4. GC deletes them.
5. Commit then updates branch to point to commit referencing deleted objects.

Result:

1. Broken history / missing objects / repository corruption.

How Git avoids this in practice:

1. Coordination/locking around refs and prune operations.
2. Grace period before pruning unreachable objects.

3. Conservative retention so recently created objects are not deleted immediately.